Chapter 1 worked solutions

DS01-01 • The upper_bound contract and proof

The precondition is that A is nondecreasing. The returned i satisfies $0 \le i \le n$, $\forall j < i, A[j] \le x$, and $\forall j \ge i, A[j] > x$, over valid indices. Initialize l=0, r=n and maintain the corresponding regions $\forall j < l, A[j] \le x$ and $\forall j \ge r, A[j] > x$. Set m=l+floor((r−l)/2). If $A[m] \le x$, set l=m+1. Otherwise set r=m. Sorted order preserves both regions. The nonnegative measure r−l strictly decreases. At l=r the invariant is the postcondition. The successive states (l, r, m) are $(0,5,2),(3,5,4),(3,4,3)$, returning 3. For an empty array l=r=0 initially, so return 0 without reading an element.

DS01-02 • Nested-loop depth is not enough

The total is $n+n/2+\ldots+1=2n-1=\Theta(n)$. If every inner loop takes n iterations, there are k+1 outer iterations. The total is $n(k+1)=n(\log_2 n+1)=\Theta(n \log n)$. The same nesting depth permits different growth rates because the inner bounds differ.

DS01-03 • Growth by a noninteger factor

Let c_s be the capacity before the last expansion. Each new capacity is at least 3/2 times the preceding one, so $c_{s-j} \le (2/3)^j c_s$. Total moves satisfy $\Sigma c_i \le c_s \Sigma (2/3)^j \le 3 c_s$. Expansion is triggered by append c_s+1, hence c_s<n and total moves are O(n). New-element writes add Θ(n). Allocating the new capacities also sums to O(n), since rounding adds only a constant per expansion and each new capacity is O(c_i). A full-array append can move Θ(n) elements and therefore has Θ(n) worst-case cost under the element-move model. The sequence has Θ(1) amortized cost per append. If no expansion occurs, the claim holds directly.

DS01-04 • Analysis types and arithmetic costs

Bitwise addition takes O(L) in the worst case, giving O(nL). If every addition processes L bits, the bound is Θ(nL). For varying lengths, the maximum L gives only an upper bound. A more precise analysis sums the effective bit lengths of all additions. Constant L can be absorbed into the constant factor. For L=n, the upper bound becomes $O(n^2)$. Constant-time RAM arithmetic assumes values fit within machine words. Arbitrary-precision arithmetic does not preserve that assumption.

DS01-05 • An explicit potential for expensive high bits

Choose $\Phi(A) = \sum_{i=0}^{k-1} (1 + 3(3/2)^i) A[i]$. Every coefficient is nonnegative and $\Phi(0) = 0$. Clearing r low ones and setting bit r costs

Chapter 1 worked solutions (continued)

$$C_r = (r+1) + \sum_{i=0}^{r} (3/2)^i = r - 1 + 3(3/2)^r.$$

The potential changes by

$$\Delta\Phi = (1 + 3(3/2)^r) - \sum_{i=0}^{r-1} (1 + 3(3/2)^i) = 7 - r - 3(3/2)^r.$$

Hence $C_r + \Delta\Phi = 6$, and T increments cost at most $6T$.

For 011→100, $r = 2$. Three reads plus writes costing $1 + 3/2 + 9/4 = 19/4$ give actual cost $31/4$. Potential changes from $a_0 + a_1 = 4 + 11/2 = 19/2$ to $a_2 = 31/4$, decreasing by $7/4$ and giving amortized cost 6. Earlier low one bits stored weighted credit for the expensive high-bit write.

With write cost 2^i, bit i is written $2^{k-i} - 1$ times before reaching all ones. The total write cost is $k2^k - (2^k - 1)$. Dividing by $2^k - 1$ operations gives $\Theta(k)$. No nonnegative potential starting at zero can turn this actual lower bound into $O(T)$ with a constant independent of k. Treating a fixed k as a machine constant is a different model.

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study